Payments, Ledger & Reconciliation

Module specification for payment orchestration and financial correctness

1. Payments and provider adapters

HLD

Payments owns attempts, provider references, authorization/capture outcomes, refunds and provider event ingestion. It does not decide customer entitlements. A provider adapter translates a stable CPG contract to the selected PSP/acquirer.

LLD / state transitions

```
created -> pending -> authorized -> succeeded
                   |          -> failed
                   |          -> cancelled
authorized -> succeeded | cancelled
succeeded -> partially_refunded -> refunded
```

- A timeout after a create/capture/refund call is uncertain. Query provider status or reconcile before retrying with a new provider operation.
- Each retry is a distinct payment attempt; never overwrite a failed attempt with the result of a new one.
- Verify webhook signature against provider-required raw bytes and timestamp window; persist event ID before acknowledging.
- Deduplicate by provider + account/tenant scope + provider event ID. Provider IDs may not be globally unique.
- Provider event ordering is not guaranteed. Apply monotonic/domain-valid transitions and fetch authoritative provider state when events conflict.
- Store only payment tokens/provider references needed for operations; never raw card data or CVV.
- Refund amount cannot exceed the captured/refundable balance after completed refunds; concurrent refund requests require atomic reservation.

Provider interface

```
export interface PaymentProvider {
  createPayment(input: CreateProviderPaymentInput): Promise<CreateProviderPaymentResult>;
  getPayment(providerPaymentId: string): Promise<ProviderPaymentSnapshot>;
  capturePayment?(input: CaptureProviderPaymentInput): Promise<CaptureProviderPaymentResult>;
  cancelPayment?(providerPaymentId: string): Promise<ProviderPaymentSnapshot>;
  refundPayment(input: RefundProviderPaymentInput): Promise<RefundProviderPaymentResult>;
  verifyWebhook(input: VerifyWebhookInput): Promise<VerifiedProviderEvent>;
}
```

Schema, DTOs and indexes

```
type PaymentDoc = {
  _id:ObjectId; tenantId:ObjectId; publicId:string; orderId:ObjectId;
  provider:string; providerAccountRef?:string; providerPaymentId?:string;
  attemptNumber:number; amountMinor:number; currency:string;
  status:"created"|"pending"|"authorized"|"succeeded"|"failed"|"cancelled"|"partially_refunded"|"refunded";
  failureCode?:string; idempotencyKey:string; createdAt:Date; updatedAt:Date;
};
// Unique {tenantId, publicId}; unique {tenantId, idempotencyKey}
// Partial unique provider reference on {tenantId, provider, providerAccountRef, providerPaymentId}
// Index {tenantId, orderId, createdAt desc}

export type CreatePaymentDTO = { orderId:string; returnUrl?:string; paymentMethodType?:string };
export type PaymentDTO = { id:string; orderId:string; provider:string; amountMinor:number;
  currency:string; status:string; failureCode?:string; createdAt:string; updatedAt:string };
export type CreateRefundDTO = { amountMinor:number; reason:"customer_request"|"duplicate"|"fraud"|"other" };
export type RefundDTO = { id:string; paymentId:string; amountMinor:number; currency:string;
  status:"pending"|"succeeded"|"failed"; reason:string; createdAt:string };
```

REST endpoints

Method / route	Behavior / status
POST /api/v1/orders/:id/payment-attempts	Create attempt; 201 created or 202 pending; Idempotency-Key required.

Method / route	Behavior / status
GET /api/v1/payments/:id	200 PaymentDTO; tenant-scoped.
POST /api/v1/payments/:id/refunds	202 RefundDTO when asynchronous; 409 if amount exceeds remaining refundable amount.
GET /api/v1/payments/:id/refunds	List refund attempts, cursor-paginated.
POST /api/v1/webhooks/:provider	Verify, persist/dedupe, enqueue; 2xx only after durable acceptance; 400 invalid signature.
POST /api/v1/operations/reconciliation-runs	Privileged operator action; 202 run ID.

UI journey

Screen	Components / states
Payment timeline	PaymentStatusBadge, AttemptList, ProviderReference (masked), EventTimeline.
Refund dialog	RefundAmountInput, RemainingRefundableSummary, ReasonSelect, ConfirmDialog; prevent double submit.
Provider configuration	ProviderStatus, SecretReference, CapabilityMatrix; never return secret values.
Reconciliation	RunHistory, ExceptionTable, ResolveExceptionDialog; every manual resolution requires reason.

Full-stack sequence

```
sequenceDiagram
  participant PSP as Provider
  participant API as Webhook API
  participant P as PaymentService
  participant DB as MongoDB
  participant Q as Outbox Worker
  participant L as LedgerService
  PSP->>API: Signed event
  API->>P: verifyAndNormalize(rawBody, headers)
  P->>DB: Insert provider event if unseen
  P->>DB: Apply valid payment transition + outbox
  API-->>PSP: 2xx after durable acceptance
  Q->>DB: Claim payment.succeeded event
  Q->>L: Post balanced financial transaction idempotently
  L->>DB: Insert ledger transaction and entries
  Q->>DB: Mark outbox event processed
```

2. Ledger and reconciliation

Use a double-entry journal for auditable financial postings. Keep customer payment success separate from provider settlement: a payment may succeed before funds settle, and provider fees, taxes, refunds, chargebacks and settlement timing create distinct entries.

Ledger invariants

- Every posted transaction balances debits and credits per currency.
- Entries are append-only. Corrections post reversals or adjustments referencing the original transaction.
- Posting uses a unique source key such as (tenantId, sourceType, sourceId, postingPurpose) to prevent duplicate journals.
- Do not treat gross customer charge as net revenue. Separate clearing, fees, taxes payable, refund liability, affiliate payable and revenue accounts.
- Ledger posting and outbox creation must be atomic within the database transaction where supported.
- Reconciliation reports unmatched CPG payments, provider payments, settlement entries, fees and refunds.

Schema

```
type LedgerTransactionDoc = {
  _id:ObjectId; tenantId:ObjectId; publicId:string;
```

```
  sourceType:string; sourceId:string; postingPurpose:string;
  status:"posted"|"reversed"; currency:string; postedAt:Date;
};
type LedgerEntryDoc = {
  _id:ObjectId; tenantId:ObjectId; transactionId:ObjectId;
  accountCode:string; direction:"debit"|"credit";
  amountMinor:number; currency:string; createdAt:Date;
};
// Unique source key on ledgerTransactions.
// Index ledgerEntries {tenantId, accountCode, createdAt}.
// Validate balanced total in service before posting; DB transaction required.
```

Reconciliation APIs

Route	Contract
GET /api/v1/reconciliation/runs	List run summaries and statuses.
POST /api/v1/reconciliation/runs	Accept provider/account/date range; return 202 run ID.
GET /api/v1/reconciliation/runs/:id/exceptions	Paginated unmatched/mismatched records.
POST /api/v1/reconciliation/exceptions/:id/resolve	Privileged resolution with enum, reason, evidence reference; audited.

UI flow: Finance Overview -> Run Reconciliation -> Exception Queue -> compare provider/CPG/ledger records -> resolve or escalate -> export report. Do not provide a generic edit field for ledger amounts.

Sequence

```
sequenceDiagram
  participant Ops as Finance UI
  participant SDK as kubera.operations
  participant API as Express
  participant R as Reconciliation Worker
  participant PSP as Provider API/Report
  participant DB as MongoDB
  Ops->>SDK: reconciliation.start(input)
  SDK->>API: POST /reconciliation/runs
  API->>DB: Save run(status=queued)
  API-->>Ops: 202 runId
  R->>PSP: Fetch transactions/settlements
  R->>DB: Compare against payments and ledger
  R->>DB: Save exceptions and final run status
  Ops->>SDK: reconciliation.get(runId)
  SDK->>API: GET /reconciliation/runs/:id
  API-->>Ops: 200 run summary
```